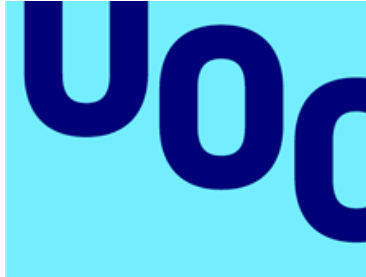


Universitat Oberta de Catalunya



PRAC 1 - Ús de bases de dades NoSQL

Autor:

Albert Lozano Valverde

email:

alozanoval@uoc.edu

Barcelona, Gener , 2026

Memòria

1 Cassandra

1.1 Exercici, modificació de dades

Primer fem la consulta

```
INSERT INTO TESTING_AGGREGATE (COLUMN1, COLUMN2, COLUMN3,
    COLUMN4) VALUES ('clave1_2', 'clave2_2', 'clave3_2',
    'valorNuevo');
```

Tornem a consultar les dades

```
SELECT * FROM TESTING_AGGREGATE;
```

Obtenim el següent resultat.

columna1	columna2	columna3	columna4
clave1_1	clave2_1	clave3_1	valor1
clave1_2	clave2_2	clave3_2	valorNuevo

Ara fem la següent consulta

```
UPDATE TESTING_AGGREGATE
    SET COLUMN4 = 'valor3'
    WHERE COLUMN1 = 'clave1_3' AND COLUMN2 = 'clave2_3' AND
        COLUMN3 = 'clave3_3';
```

Tornem a consultar les dades i aquesta vegada obtenim

columna1	columna2	columna3	columna4
clave1_1	clave2_1	clave3_1	valor1
clave1_2	clave2_2	clave3_2	valorNuevo
clave1_3	clave2_3	clave3_3	valor3

Per què ha passat això? Quines diferències hi ha entre aquest comportament i el d'una base de dades relacional o tradicional?

A Cassandra, un INSERT no comprova si el registre ja existeix. Si la combinació de partition key + clustering columns (COLUMN1, COLUMN2, COLUMN3) ja existeix, Cassandra sobreescrui el valor anterior de les columnes indicades. Per tant el valor valor2 és substituït per valorNuevo. Com a resultat no apareix cap fila duplicada, ni es produeix cap error de clau duplicada.

Aquest comportament es coneix com upsert (update or insert). El mateix comportament es veu en la segona consulta la clau primària no existeix, per tant Cassandra crea automàticament la fila amb els valors indicats. Per tant apareix una nova fila amb clave1_3 / clave2_3 / clave3_3, i el valor valor3 queda emmagatzemat encara que abans no hi hagués cap registre.

Aquest comportament és intencionat i necessari per garantir rendiment i escalabilitat en entorns distribuïts. A diferència d'una BD relacional, Cassandra prioritza la disponibilitat i el rendiment per sobre de les restriccions i comprovacions d'integritat.

1.2 Consulta 1

Tal com hem vist, hi ha hagut algun problema en la càrrega de dades del fitxer de viatges en bicicleta. La taula TRIPS hauria de tenir 124.954 registres, però només en té 43.049. Explica per què la inserció s'ha realitzat de forma errònia, fes una càrrega de dades que permeti carregar els registres de forma correcta i posa un exemple d'un registre que no s'hagi carregat la primera ocasió i explica'n el perquè.

El motiu es que al crear la taula hem definit com a única clau primària el ID_User però cada usuari pot fer múltiples viatges per tant al fer un altre viatge es sobreescriu la entrada perquè coincideixen les claus primàries pel comportament que hem indicat abans, per solucionar-ho al crear la taula podem indicar un altre variable com id, en concret crec que DATE es la indicada perquè un usuari farà un viatge en un moment. Per tant la construcció ens quedaria de la següent manera.

```
CREATE TABLE TRIPS (  
  ID_USER TEXT,  
  EMAIL TEXT,  
  CREDITS DECIMAL,  
  FROM_STATION INT,  
  TO_STATION INT,  
  BIKE TEXT,  
  DATE TIMESTAMP,  
  KILOMETERS DECIMAL,  
  PRIMARY KEY (ID_USER, DATE));
```

Per tant fem un drop a la taula per tornar-la a crear amb aquestes condicions i carregen les dades. Si ara tornem a fer la consulta per mirar el numero de files que tenim obtenim

count
124954

1.3 Consulta 2

Ens demanen crear una consulta per veure si hi ha diferències en el nombre de viatges i quilòmetres realitzats per les bicicletes de les marques BH i Orbea abans que es facin malbé (estat = 'out'). La consulta ha de tornar el nombre de viatges i el nombre de quilòmetres de les bicicletes espatllades agrupats pel nom de la seva marca.

A bikes.csv ja tenim brand, status, rides (nombre de viatges acumulats per bici) i kms (km acumulats per bici). Per tant, no ens cal TRIPS per aquesta consulta; és molt més òptim utilitzar BIKES (menys volum, menys agregats). Primer creem un agregat ja sumat per marca+estat

```
CREATE TABLE IF NOT EXISTS bike_totals_by_brand_status (  
    brand text,  
    status text,  
    total_rides bigint,  
    total_kms decimal,  
    PRIMARY KEY ((brand, status)));
```

Ara generem un CSV agregat des de bikes.csv per fer-ho fem la següent comanda desde linux

```
python3 - <<'PY'  
import csv  
from decimal import Decimal  
  
inp = "/home/student/Escritorio/Cassandra/bikes.csv"  
out =  
    "/home/student/Escritorio/Cassandra/bike_totals_by_brand_status.csv"  
  
tot = {} # (brand,status) -> [rides_sum, kms_sum]  
with open(inp, newline='') as f:  
    r = csv.DictReader(f)  
    for row in r:  
        brand = row["brand"]  
        status = row["status"]  
        rides = int(row["rides"])  
        kms = Decimal(row["kms"])  
        key = (brand, status)  
        if key not in tot:  
            tot[key] = [0, Decimal("0")]  
        tot[key][0] += rides  
        tot[key][1] += kms  
  
with open(out, "w", newline="") as g:  
    w = csv.writer(g)  
    w.writerow(["brand", "status", "total_rides", "total_kms"])  
    for (brand,status),(rs,ks) in tot.items():  
        w.writerow([brand,status,rs,str(ks)])  
  
print("Wrote", out, "rows:", len(tot))  
PY
```

Ara podem carregar les dades des de el csv a Cassandra

```
COPY bike_totals_by_brand_status (brand, status, total_rides,
total_kms)
FROM
    '/home/student/Escritorio/Cassandra/bike_totals_by_brand_status.csv'
WITH DELIMITER=',' AND HEADER=TRUE;
```

Per tant podem fer les consultes següents per obtenir el que volíem

```
SELECT brand, total_rides, total_kms
FROM bike_totals_by_brand_status
WHERE brand='BH' AND status='out';

SELECT brand, total_rides, total_kms
FROM bike_totals_by_brand_status
WHERE brand='Orbea' AND status='out';
```

El resultat d'aquesta consulta es el següent

brand	total rides	total kms
BH	4052	12125.823933287104888
Orbea	4679	14091.599613048982984

1.4 Consulta 3

Suposa que diferents ciutats han contractat la nostra empresa per gestionar les seves bicicletes. En conseqüència, s'espera una quantitat de dades molt més gran i, per poder-les gestionar eficientment, es proposa crear un sistema que permeti repartir la càrrega de les dades de les bicicletes en 10 nodes de forma distribuïda. Analitza si l'agregat que has creat a la consulta anterior seria eficient en aquest nou entorn. En cas contrari, proposa un nou agregat que es comporti de forma més eficient. Justifica la teva resposta.

L'agregat de Consulta 2 amb (brand,status) té poques particions per tant risc de hot partitions i contenció. Podríem utilitzar `geo_cell` (derivada de `location.coordinates` aproximant depenent de la distància entre les ciutats) augmenta cardinalitat i en conseqüència millorant distribució i consultes locals. Un altre opció seria afegir `bucket` permet repartir encara més la càrrega, aproximant el repartiment sobre 10 nodes. On $bucket = hash(bike_id) \% 10$ (o $\% N$). Per tant les taules en cada cas tindrien el següent aspecte, en funció si volem utilitzar el `bucket` o no.

```
CREATE TABLE IF NOT EXISTS bikes_by_geo_brand_status (
    geo_cell text,
    brand text,
    status text,
    bike_id text,
```

```

PRIMARY KEY ((geo_cell, brand, status), bike_id)
);

CREATE TABLE IF NOT EXISTS bikes_by_geo_brand_status_bucket (
    geo_cell text,
    brand text,
    status text,
    bucket int,
    bike_id text,
    PRIMARY KEY ((geo_cell, brand, status, bucket), bike_id)
);

```

1.5 Consulta 4

La nostra empresa vol repintar les bicicletes que tinguin ratllades (damages = scratches). Com que de moment no ha comprat pintura per a totes, començarà pintant les bicicletes de la marca Orbea que estiguin disponibles (status = free). Per fer-ho, ens demanen crear una consulta, que puguin executar cada dia, per identificar les matrícules de les bicicletes de la marca Orbea que estiguin lliures.

Amb la taula BIKES actual no podem (sense ALLOW FILTERING), perquè damages CONTAINS + brand + status no són PK. Per tant hem de fer una taula índex per damage+brand+status de la següent manera

```

CREATE TABLE IF NOT EXISTS bikes_by_damage_brand_status (
    damage text,
    brand text,
    status text,
    bike_id text,
    PRIMARY KEY ((damage, brand, status), bike_id)
);

```

Després generarem les dades per poder fer la consulta a partir d'un script de python que correrem a una terminal a la carpeta per generar el csv necessari.

```

python3 - <<'PY'
import csv, ast

inp = "/home/student/Escritorio/Cassandra/bikes.csv"
out =
    "/home/student/Escritorio/Cassandra/bikes_by_damage_brand_status.csv"

with open(inp, newline='') as f, open(out, 'w', newline='') as g:
    r = csv.DictReader(f)

```

```

w = csv.DictWriter(g,
    fieldnames=["damage", "brand", "status", "bike_id"])
w.writeheader()
for row in r:
    bike_id = row["_id"]
    brand = row["brand"]
    status = row["status"]
    damages_raw = row.get("damages") or "[]"
    try:
        damages = ast.literal_eval(damages_raw)
    except Exception:
        damages = []
    for d in damages:
        w.writerow({"damage": d, "brand": brand, "status": status,
            "bike_id": bike_id})

print("Wrote", out)
PY

```

Ara podem carregar les dades des de el csv a Cassandra

```

COPY bikes_by_damage_brand_status (damage, brand, status, bike_id)
FROM
    '/home/student/Escritorio/Cassandra/bikes_by_damage_brand_status.csv'
WITH DELIMITER=',' AND HEADER=TRUE;

```

Per tant podem fer les consultes següents per obtenir el que volíem

```

SELECT bike_id
FROM bikes_by_damage_brand_status
WHERE damage='scratches' AND brand='Orbea' AND status='free';

```

El resultat d'aquesta consulta es el següent

bike_id
78TZOL5ETI
DLLJ4SF15U
GG3R5IHOP8
RGZT06ZJ2S
S4P48497T6
T5XLUSN2MJ
XG25P05KC5

1.6 Consulta 5

Per entendre millor els patrons de viatge dels usuaris, ens demanen una consulta que permeti identificar el nombre de viatges de tots els usuaris

per a cada parell d'estacions d'origen i destí. En particular, la consulta haurà de permetre escollir un parell d'estacions d'origen i destí i mostrar el nombre de viatges realitzats entre aquestes estacions, la mitjana de quilòmetres dels viatges les estacions i la suma de crèdit consumit.

Primer generem la taula la qual tindrà com a id les dos estacions:

```
CREATE TABLE IF NOT EXISTS trip_stats_by_station_pair (  
    from_station int,  
    to_station int,  
    n_trips bigint,  
    avg_kms decimal,  
    total_credits decimal,  
    PRIMARY KEY ((from_station, to_station))  
);
```

Agrupem les claus de la forma necessària i els passem a csv

```
python3 - <<'PY'  
import csv  
from decimal import Decimal  
  
inp = "/home/student/Escritorio/Cassandra/viajes.csv"  
out =  
    "/home/student/Escritorio/Cassandra/trip_stats_by_station_pair.csv"  
  
agg = {} # (from,to) -> [count, total_kms, total_credits]  
  
with open(inp, newline='') as f:  
    r = csv.DictReader(f)  
    for row in r:  
        fs = int(row["from"])  
        ts = int(row["to"])  
        kms = Decimal(str(row["kms"])) # kms float -> Decimal (via str)  
        credits = Decimal(str(row["credits"])) # credits -> Decimal  
        key = (fs, ts)  
        if key not in agg:  
            agg[key] = [0, Decimal("0"), Decimal("0")]  
        agg[key][0] += 1  
        agg[key][1] += kms  
        agg[key][2] += credits  
    for key in agg:  
        agg[key][1] = agg[key][1]/agg[key][0]  
    with open(out, "w", newline="") as g:  
        w = csv.writer(g)  
        w.writerow(["from_station", "to_station", "n_trips", "avg_kms", "total_credits"])  
        for (fs, ts), (cnt, tk, tc) in agg.items():  
            w.writerow([fs, ts, cnt, str(tk), str(tc)])
```



```
print("Wrote", out, "rows:", len(agg))
PY
```

Ara podem carregar les dades des de el csv a Cassandra

```
COPY trip_stats_by_station_pair (from_station, to_station, n_trips,
    total_kms, total_credits)
FROM
    '/home/student/Escritorio/Cassandra/trip_stats_by_station_pair.csv'
WITH DELIMITER=',' AND HEADER=TRUE;
```

Per tant podem fer les consultes següents per obtenir el que volíem

```
SELECT n_trips, avg_kms, total_credits
FROM trip_stats_by_station_pair
WHERE from_station=40 AND to_station=60;
```

El resultat d'aquesta consulta es el següent

n_trips	avg_kms	total_credits
44	54.635272727272729772727273	18.48000000000000022

La mitjana es una mica alta, he explorat las dades i he vist que hi ha outliners com distancies que posen 529, suposo que son metres i no s'ha fet be la conversió a kilometres.

2 Neo4j

2.1 Consulta 1

Mostra per pantalla el recompte de naus que apareixen a la pel·lícula 'A New Hope', de manera que, en la primera columna, aparegui el nombre de naus i, en la segona, una llista amb els noms d'aquestes naus.

Identifiquem del film: MATCH (f:Film name: 'A New Hope') filtra exactament la pel·lícula requerida a partir de la seva propietat name, despres recuperem les naus associades al film: el patró (s:Starship)-[:APPEARS_IN]->(f) explotant el disseny de grafs: les "naus que apareixen" estan connectades directament al film. Per evitar duplicats: collect(DISTINCT s.name) ja que una nau pot quedar relacionada més d'un cop (per càrrega de dades o múltiples aparicions). Per ultim queda el recompte i llista en una sola fila: Utilitzem collect(...) genera la llista de noms i size(nomsNaus) que dona el recompte. Per tant la consulta serà

```
MATCH (f:Film {name: 'A New Hope'})
MATCH (s:Starship)-[:APPEARS_IN]->(f)
WITH collect(DISTINCT s.name) AS nomsNaus
```

```

RETURN
  size(nomsNaus) AS nombreNaus,
  nomsNaus      AS llistaNaus;

```

El resultat de la consulta és

nombreNaus	llistaNaus
5	["Millennium Falcon", "X-wing", "Tantive IV", "X-34 Landspeeder", "TIE Advanced x1"]

2.2 Consulta 2

Mostra per pantalla el planeta del qual procedeixen el nombre més gran de personatges de la nostra base de dades indicant, en la primera columna, el nom del planeta, en la segona la quantitat de personatges nascuts allà i, finalment, en la tercera columna, la llista amb els seus noms.

El personatges tenen la relació (:Character)-[:IS_HOMEWORLD]->(:Planet), així que només hem de recorre el graf. Agrupem per planeta: el WITH p, ... agrupa tots els personatges per cada planeta. Hem de generar la llista i el recompte consistents: fem collect(DISTINCT c.name) per obtenir la llista i després size(...) per comptar exactament els mateixos elements. Per últim fem selecció del màxim: ORDER BY ... DESC LIMIT 1 retorna el planeta amb el recompte més alt.

Per tant la consulta serà

```

MATCH (c:Character)-[:IS_HOMEWORLD]->(p:Planet)
WITH
  p,
  collect(DISTINCT c.name) AS nomsPersonatges
RETURN
  p.name          AS planeta,
  size(nomsPersonatges) AS quantitatPersonatges,
  nomsPersonatges AS llistaPersonatges
ORDER BY quantitatPersonatges DESC
LIMIT 1;

```

El resultat de la consulta és

planeta	quantitatPersonatges	llistaPersonatges
"Tatooine"	6	["Luke Skywalker", "Anakin Skywalker", "C-3PO", "Darth Vader", "Biggs Darklighter", "Shmi Skywalker"]

2.3 Consulta 3

A la següent consulta volem obtenir els noms de les espècies els personatges de les quals provenen de més d'un planeta. A la primera columna es mostrarà el nom de l'espècie que compleix aquest requisit, a la segona, el nombre de planetes total del qual prové el conjunt dels seus individus i, a

la tercera, el llistat d'aquests planetes. Han d'aparèixer en primer lloc les espècies el conjunt d'individus de les quals pertanyin a més planetes.

Connectem espècie i personatges amb la relació Character IS_OF_SPECIE Species permet agrupar individus per espècie. Utilitzant la relació que hem vist abans Character IS_HOMEWORLD Planet obtenim el planeta de procedència de cada individu. Com sempre per evitar duplicats farem collect(DISTINCT p.name) assegurant que un planeta només compta un cop per espècie, encara que hi hagi molts individus del mateix planeta. Ara filtre, "més d'un planeta": WHERE numPlanetes > 1 implementa el requisit literal. Per últim ordenem utilitzant ORDER BY numPlanetes DESC fa que surtin primer les espècies amb individus provinents de més planetes.

Per tant la consulta serà

```
MATCH (c:Character)-[:IS_OF_SPECIE]->(s:Species)
MATCH (c)-[:IS_HOMEWORLD]->(p:Planet)
WITH
    s,
    collect(DISTINCT p.name) AS planetes
WITH
    s.name AS especie,
    planetes,
    size(planetes) AS numPlanetes
WHERE numPlanetes > 1
RETURN
    especie,
    numPlanetes,
    planetes AS llistatPlanetes
ORDER BY numPlanetes DESC, especie ASC;
```

El resultat de la consulta és

especie	numPlanetes	llistatPlanetes
"Human"	28	["Stewjon", "LLothal", "Glee Anselm", "Alderaan", "Tatooine", "Axxila", "Jedha", "Corellia", "Coruscant", "Parnassos", "Naboo", "Serenno", "Jakku", "Vallt", "Lexrul", "Hays Minor", "Mandalore", "Socorro", "Yavin 4", "Kamino", "Fest", "Concord Dawn", "Arkanis", "Chandрила", "Haruun Kal", "Grange", "Bespin", "Onderon"]
"Droid"	2	["Naboo", "Tatooine"]

2.4 Consulta 4

Volem obtenir els personatges que van pilotar alguna nau que apareix a una pel·lícula el títol de la qual contingui, sigui en majúscules o sigui minúscules, la paraula Hope. Per cada resultat s'ha de mostrar el títol de la pel·lícula, el nom del personatge i el nom de la nau. Els resultats estaran

ordenats primer pel títol de la pel·lícula, a continuació pel nom del personatge i finalment pel nom de la nau.

Filtrem de manera case-insensitive per tant `toLower(f.name) CONTAINS 'hope'` garanteix que coincideix amb "Hope", "HOPE", "hOpE", etc. Restringim a naus que apareixen en el film utilitzant el patró `(s)-[:APPEARS_IN]->(f)` assegurant que la nau està associada a aquella pel·lícula. Després mirem el pilotatge del personatge utilitzant la relació `(c)-[:PILOTED]->(s)` que enllaça el personatge amb la nau que ha pilotat. Per últim es retorna només el necessari i `ORDER BY` compleix l'ordre requerit (film → personatge → nau).

Per tant la consulta serà

```
MATCH (f:Film)
WHERE toLower(f.name) CONTAINS 'hope'
MATCH (s:Starship)-[:APPEARS_IN]->(f)
MATCH (c:Character)-[:PILOTED]->(s)
RETURN
    f.name AS titolPelícula,
    c.name AS nomPersonatge,
    s.name AS nomNau
ORDER BY
    titolPelícula ASC,
    nomPersonatge ASC,
    nomNau ASC;
```

El resultat de la consulta és

titolPelícula	nomPersonatge	nomNau
"A New Hope"	Chewbacca	"Millennium Falcon"
"A New Hope"	"Darth Vader"	"TIE Advanced x1"
"A New Hope"	"Han Solo"	"Millennium Falcon"
"A New Hope"	L:Leia Organa	"Tantive IV"
"A New Hope"	L:Luke Skywalker	"X-34 Landspeeder"
"A New Hope"	L:Luke Skywalker	"X-wing"

2.5 Consulta 5

Ens sol·liciten el llistat dels personatges de l'espècie 'Human' que han pilotat més d'una nau del tipus 'Patrol craft'. Per a això, en la primera columna ha d'aparèixer el nom del personatge, en la segona el nombre de naus diferents d'aquesta classe que ha pilotat, en la tercera la llista amb els noms d'aquestes naus.

Filtrem per espècie `(:Specie name:'Human')` utilitzant relació `IS_OF_SPECIE` per restringir a personatges humans. Utilitzant la relació `(:Character)-[:PILOTED]->(:Starship)` podem identifica les naus pilotades. Utilitzant la re-

lació [:STARSHIP_CLASS] podem restringir a la classe objectiu fent (:Class name: 'Patrol craft'). Fem collect(DISTINCT s.name) evita duplicats si el mateix pilotatge apareix repetit. Filtrem quan tenim més d'una amb WHERE numNaus > 1 implementa el requisit.

Per tant la consulta serà

```
MATCH (c:Character)-[:IS_OF_SPECIE]->(s:Species {name: 'Human'})
MATCH (c)-[:PILOTED]->(s:Starship)-[:STARSHIP_CLASS]->(c2:Class {name:
'Patrol craft'})
WITH
  c,
  collect(DISTINCT s.name) AS nausPatrol
WITH
  c.name AS personatge,
  nausPatrol,
  size(nausPatrol) AS numNaus
WHERE numNaus > 1
RETURN
  personatge,
  numNaus,
  nausPatrol AS llistaNaus
ORDER BY numNaus DESC, personatge ASC;
```

El resultat de la consulta és

titolPelicula	nomPersonatge	nomNau
personatge	numNaus l	listaNaus
"Boba Fett"	2	["Slave 1", "Slave II"]

2.6 Consulta 6

Volem llistar una classificació dels personatges de Star Wars que han acabat pilotant determinades naus espacials. En concret, es pretén catalogar els personatges de la facció 'Pilot Rebel', que hagin pilotat almenys una nau el nom de la qual contingui X-wing, o bé com a 'Pilot Imperial', en cas d'haver pilotat alguna nau el nom de la qual inclogui TIE. La consulta ha de mostrar, en la primera columna el nom del personatge, en la segona la seva categoria de pilot (segons les dues denominacions indicades anteriorment), en la tercera el nombre total de naus que ha pilotat i, finalment, la llista amb els noms d'aquestes naus. El resultat final ha d'excloure els personatges que no entrin en cap d'aquestes dues categories, i ordenar-se de major a menor segons el nombre total de naus (sigui el que sigui el nom de la nau) pilotades pel personatge.

Comencem amb l'agregació de naus pilotades per personatge, utilitzant collect(DISTINCT sAll.name) creem la llista completa de naus pilotades i evitant duplicats. Això permet calcular el total de naus pilotades (size(totesLesNaus)), i fer comprovacions de text sobre els noms. Ara detectem patrons als noms

(case-insensitive) utilitzant any(... toLower(n) CONTAINS 'x-wing') i any(... CONTAINS 'tie') determinen si el personatge entra en algun criteri. Ara utilitzat CASE assignem una de les dues etiquetes requerides i deixa NULL els que no encaixen. Només queda exclir dels que no encaixen fent WHERE categoria IS NOT NULL. Per últim Ordenem seguin el criteri ORDER BY totalNausPilotades DESC ordena de més a menys segons el total de naus pilotades (independentment del nom). Per tant la consulta serà

```
MATCH (c:Character)-[:PILOTED]->(sAll:Starship)
WITH c, collect(DISTINCT sAll.name) AS totesLesNaus
WITH
  c,
  totesLesNaus,
  any(n IN totesLesNaus WHERE toLower(n) CONTAINS 'x-wing') AS
    haPilotatXWing,
  any(n IN totesLesNaus WHERE toLower(n) CONTAINS 'tie') AS
    haPilotatTIE
WITH
  c,
  totesLesNaus,
  haPilotatXWing,
  haPilotatTIE
WITH
  c,
  totesLesNaus,
  CASE
    WHEN haPilotatXWing THEN 'Pilot Rebel'
    WHEN haPilotatTIE THEN 'Pilot Imperial'
    ELSE NULL
  END AS categoria
WHERE categoria IS NOT NULL

RETURN
  c.name          AS personatge,
  categoria        AS categoriaPilot,
  size(totesLesNaus) AS totalNP,
  totesLesNaus     AS llistaNaus
ORDER BY
  totalNP DESC;
```

El resultat de la consulta és

personatge	categoriaPilot	totalNP	llistaNaus
L:Luke Skywalker"	"Pilot Rebel"	3	["X-wing", "X-34 Landspeeder", "Snowspeeder"]
"Darth Vader"	"Pilot Imperial"	2	["Executor", "TIE Advanced x1"]

3 MongoDB

3.1 Consulta 1

Per a una campanya de revisió d'hivern es busca un tipus de bicicleta molt específic: bicicletes de la marca "Orbea" o "BH" i que, a més, incloguin les funcionalitats "absBrakes" i "lamps" (poden tenir més funcionalitats, però han d'incloure aquestes dues). Cal mostrar la seva matrícula, marca i les seves característiques i ordenar els resultats per la matrícula en ordre ascendent. Per a aquesta consulta, mostreu el nombre de documents obtinguts i un exemple dels 5 primers.

Per fer la primera part de la consulta utilitzarem `countDocuments` per contar el numero d'entrades que tenim. Per trobar-les primer filtrarem per brand, que ha de ser de la llista ["Orbea", "BH"] i ha de tenir les característiques ["absBrakes", "lamps"], per obtenir les entrades utilitzarem `find` i `limit(5)`.

Per tant les consultes seran

```
db.bikes.countDocuments({
  brand: { $in: ["Orbea", "BH"] },
  extraFeatures: { $all: ["absBrakes", "lamps"] }
})
```

i

```
db.bikes.find(
{
  brand: { $in: ["Orbea", "BH"] },
  extraFeatures: { $all: ["absBrakes", "lamps"] }
},
{
  _id: 1,
  brand: 1,
  extraFeatures: 1
}
).sort({ _id: 1 }).limit(5).pretty()
```

i els resultats seran

```

1 480
2
3 {
4   "_id" : "034N2U4KZ7",
5   "brand" : "BH",
6   "extraFeatures" : [
7     "lamps",
8     "helmetDetection",
9     "locker",
10    "absBrakes"
11  ]
12 }
13 {
14   "_id" : "04AG5FNEKM",
15   "brand" : "Orbea",
16   "extraFeatures" : [
17     "lamps",
18     "helmetDetection",
19     "absBrakes",
20     "locker"
21  ]
22 }
23 {
24   "_id" : "04YWRCPRM9",
25   "brand" : "BH",
26   "extraFeatures" : [
27     "helmetDetection",
28     "locker",
29     "absBrakes",
30     "lamps"
31  ]
32 }
33 {
34   "_id" : "0A215FW2R1",
35   "brand" : "BH",
36   "extraFeatures" : [
37     "helmetDetection",
38     "lamps",
39     "absBrakes",
40     "locker"
41  ]
42 }
43 {

```



```

44   "_id" : "0AY9PVAS60",
45   "brand" : "Orbea",
46   "extraFeatures" : [
47     "locker",
48     "lamps",
49     "absBrakes",
50     "helmetDetection"
51   ]
52 }

```

3.2 Consulta 2

Recuperar totes les bicicletes que tinguin almenys dos danys reportats i que, a més a més, tinguin un casc o menys. Mostrar només la matrícula (és a dir, el `_id`), la llista de danys i el nombre de cascos disponibles. Ordenar els resultats en ordre descendent pel nombre de danys, de cascos disponibles i `_id` (en aquest ordre). Per a aquesta consulta, adjunteu el nombre de documents obtinguts al realitzar la consulta i un exemple dels 5 primers.

Farem de la mateixa manera que l'apartat anterior amb dos consultes. Per filtrar hem de tenir les condicions almenys 2 danys per tant `$expr + $size(damages) >= 2` i 1 casc o menys per tant helmets: `$lte: 1` Mostrel només `_id`, `damages` i `helmets`.

Per tant les consultes seran

```

db.bikes.countDocuments({
  $expr: { $gte: [ { $size: { $ifNull: ["$damages", []] } }, 2 ] },
  helmets: { $lte: 1 }
})

```

i

```

db.bikes.aggregate([
  {
    $match: {
      $expr: { $gte: [ { $size: { $ifNull: ["$damages", []] } }, 2 ] },
    },
    helmets: { $lte: 1 }
  },
  {
    $addFields: {
      nDamages: { $size: { $ifNull: ["$damages", []] } }
    }
  },
  {

```

```

    $sort: { nDamages: -1, helmets: -1, _id: -1 }
  },
  {
    $project: {
      _id: 1,
      damages: 1,
      helmets: 1
    }
  },
  { $limit: 5 }
}).pretty()

```

i els resultats seran

```

1  69
2
3  {
4    "_id" : "ZY4ZM45EWI",
5    "helmets" : 1,
6    "damages" : [
7      "scratches",
8      "flatTire"
9    ]
10 }
11 {
12   "_id" : "Z2PHKZ9Z1K",
13   "helmets" : 1,
14   "damages" : [
15     "scratches",
16     "flatTire"
17   ]
18 }
19 {
20   "_id" : "XL1AEMV8B7",
21   "helmets" : 1,
22   "damages" : [
23     "scratches",
24     "flatTire"
25   ]
26 }
27 {
28   "_id" : "XIWZMEH3V6",
29   "helmets" : 1,
30   "damages" : [
31     "brokenLamp",

```

```

32     "flatTire"
33   ]
34 }
35 {
36   "_id" : "WICKRWVPR0",
37   "helmets" : 1,
38   "damages" : [
39     "scratches",
40     "stolenHelmet"
41   ]
42 }

```

3.3 Consulta 3

L'equip de màrqueting està preparant la segmentació d'usuaris per a la propera temporada i necessiten entendre la distribució d'edat dels nostres usuaris. Per això sol·liciten un informe que agrupi els usuaris pel seu any de naixement, és a dir, necessiten obtenir un recompte de quants usuaris han nascut cada any. S'haurà de mostrar els anys de naixement i el nombre total d'usuaris nascuts per cada any, ordenats per any ascendent. Limitar el nombre de resultats a cinc

Primer calcularem l'any de naixement a partir de la variable \$birthDate. Creem un camp nou birthYear amb l'any extret del camp birthDate (que és un Date). Agrupem per any i comptem el numero d'usuaris que tenen el mateix birthYear. Utilitzant totalUsers: { \$sum: 1 } tenim un comptador per cada usuari del grup. Ara trèiem l'_id intern del resultat (_id: 0) i reanomenem el camp _id (que era l'any) a year, mantenint totalUsers. Per ultim ordenem per any ascendentment i limitem a 5 Per tant la consulta serà

```

db.users.aggregate([
  {
    $project: {
      birthYear: { $year: "$birthDate" }
    }
  },
  {
    $group: {
      _id: "$birthYear",
      totalUsers: { $sum: 1 }
    }
  },
  {
    $project: {
      _id: 0,

```

```

        year: "$_id",
        totalUsers: 1
      }
    },
    { $sort: { year: 1 } },
    { $limit: 5 }
  ]).pretty()

```

i els resultats seran

```

1 { "totalUsers" :82,"year" :1959}
2 { "totalUsers" :580,"year" :1960}
3 { "totalUsers" :586,"year" :1961}
4 { "totalUsers" :586,"year" :1962}
5 { "totalUsers" :568,"year" :1963}

```

3.4 Consulta 4

L'equip d'operacions vol entendre millor els patrons d'ús. Sol·liciten un informe que mostri la distància mitjana dels viatges recents, segmentat per grup d'edat, amb l'objectiu d'optimitzar la logística de les bicicletes. Calculeu la distància mitjana dels viatges dels usuaris, segmentant-los per grups d'edat. Els grups han de ser: [16-25], [26-35], [36-45], [46-55] i [56-65]. SUGGERIMENT: per a la resolució d'aquesta consulta cal utilitzar, entre d'altres, l'etapa d'agregació `$bucket`.

Primer fem un match inicial per limita a usuaris 16–65 i amb rides. Continuem amb unwind que converteix l'array rides en registres individuals. Ara com ha s'ha suggerit utilitzarem bucket per crear els 5 intervals d'edat i calcular avgKms per interval. Acabem amb project per posar el rang en format [xx-yy] i mostra també totalRides. Per tant la consulta serà

```

db.users.aggregate([
  {
    $match: {
      age: { $gte: 16, $lte: 65 },
      "rides.kms": { $exists: true }
    }
  },
  { $unwind: "$rides" },
  {
    $match: {
      "rides.kms": { $type: "number" }
    }
  },
  {

```

```

$bucket: {
  groupBy: "$age",
  boundaries: [16, 26, 36, 46, 56, 66],
  default: "Fora de rang",
  output: {
    totalRides: { $sum: 1 },
    avgKms: { $avg: "$rides.kms" }
  }
},
{
  $project: {
    _id: 0,
    ageGroup: {
      $switch: {
        branches: [
          { case: { $eq: ["$_id", 16] }, then: "[16-25]" },
          { case: { $eq: ["$_id", 26] }, then: "[26-35]" },
          { case: { $eq: ["$_id", 36] }, then: "[36-45]" },
          { case: { $eq: ["$_id", 46] }, then: "[46-55]" },
          { case: { $eq: ["$_id", 56] }, then: "[56-65]" }
        ],
        default: "$_id"
      }
    },
    totalRides: 1,
    avgKms: 1
  }
},
{ $sort: { ageGroup: 1 } }
]).pretty()

```

i els resultats seran

```

1 {
2   "totalRides" :33826,
3   "avgKms" :3.00813217643233,
4   "ageGroup" :"[16-25]"
5 }
6 {
7   "totalRides" :29075,
8   "avgKms" :3.0043853482373173,
9   "ageGroup" :"[26-35]"
10 }
11 {
12   "totalRides" :29978,
13   "avgKms" :3.0119479618386817,

```

```

14   "ageGroup" : "[36-45]"
15 }
16 {
17   "totalRides" : 11784,
18   "avgKms" : 3.0074741174473862,
19   "ageGroup" : "[46-55]"
20 }
21 {
22   "totalRides" : 11766,
23   "avgKms" : 3.0034864864864863,
24   "ageGroup" : "[56-65]"
25 }

```

3.5 Consulta 5

Es vol crear un índex d'antiguitat, Index, per a les bicicletes que ens ajudi a identificar les més usades. Aquest índex es calcula a partir de la fórmula següent: $(kms \times 0.4) + (rides \times 0.6)$. Calculeu aquest índex per a totes les bicicletes de la marca "Vandor" que no estiguin netes. Mostrar la matrícula, kms, rides, i el Index calculat, ordenat de major a menor antiguitat, és a dir, en ordre descendent i mostrant només els 5 primers resultats.

SUGGERIMENT: per la resolució d'aquesta consulta cal utilitzar, entre d'altres operadors i etapes, els operadors aritmètics. Es recomana llegir la documentació proporcionada.

Primer fem un match per filtrar la marca Vandor i els bruts. Després utilitzant \$addField creem el camp calculat Index amb \$multiply i \$add. Mostrem només els camps demanats amb \$project. I ordenem i limitem a 5. Per tant la consulta serà

```

db.bikes.aggregate([
  {
    $match: {
      brand: "Vandor",
      isClean: false
    }
  },
  {
    $addField: {
      Index: {
        $add: [
          { $multiply: ["$kms", 0.4] },
          { $multiply: ["$rides", 0.6] }
        ]
      }
    }
  }
])

```

```

    }
  },
  {
    $project: {
      _id: 1,
      kms: 1,
      rides: 1,
      Index: 1
    }
  },
  { $sort: { Index: -1, _id: 1 } },
  { $limit: 5 }
]).pretty()

```

i els resultats seran

```

1 {
2   "_id" : "NS8ZLP0XT1",
3   "rides" : 31,
4   "kms" : 96.13801120544827,
5   "Index" : 57.055204482179306
6 }
7 {
8   "_id" : "84XEIYRXIK",
9   "rides" : 32,
10  "kms" : 94.130549200319,
11  "Index" : 56.852219680127604
12 }
13 {
14  "_id" : "N9IVQEB4AG",
15  "rides" : 30,
16  "kms" : 95.94129102933124,
17  "Index" : 56.376516411732496
18 }
19 {
20  "_id" : "SZ7D3PZQ9T",
21  "rides" : 31,
22  "kms" : 94.17094292840434,
23  "Index" : 56.268377171361735
24 }
25 {
26  "_id" : "ZORF9ESTLF",
27  "rides" : 30,
28  "kms" : 94.0678170684848,
29  "Index" : 55.62712682739392

```

30 }

3.6 Consulta 6

El departament de manteniment ha detectat que l'accessori l'amps"està donant problemes elèctrics i ha decidit retirar físicament aquest accessori de totes les bicicletes. Com a conseqüència, hem d'actualitzar la base de dades. Per tant, cal eliminar la funcionalitat l'amps"de l'array de funcionalitats extra de totes les bicicletes que la tinguin instal·lada. En aquest cas cal adjuntar la consulta que modifica les dades, el que retorna i una altra consulta que mostri que les dades han quedat correctament modificades, és a dir, que no hi ha resultats si es busquen bicicletes amb "lamps".

Primer editar el que farem serà buscar els que tenen extraFeatures lamps i extreure-les utilitzant la funció \$pull Per tant tindrem la següent consulta

```
db.bikes.updateMany(  
  { extraFeatures: "lamps" },  
  { $pull: { extraFeatures: "lamps" } }  
)
```

i els resultats seran

```
1 {  
2   "acknowledged" :true,  
3   "matchedCount" :995,  
4   "modifiedCount" :995  
5 }
```

Ara buscarem quants queden que tinguin extraFeatures lamps per comprovar que els hem eliminat tots. Per tant tindrem la següent consulta

```
db.bikes.countDocuments({ extraFeatures: "lamps" })
```

i els resultats seran

```
1 0
```

Per tant hem borrrat tots